

FAKE JOB DETECTION
A PROJECT REPORT

Submitted By

J.EBSHIBHA (922022104007)

In partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

SRI VIDYA COLLEGE OF ENGINEERING & TECHNOLOGY
VIRUDHUNAGAR – 626 005



ANNA UNIVERSITY : CHENNAI-600 025

MAY 2026

ANNA UNIVERSITY : CHENNAI-600 025

BONAFIDE CERTIFICATE

Certified that this Project report titled “ **FAKE JOB DETECTION** ” is the bonafide work of “ **J.EBSHIBHA (922022104007)** ” who carried out the Project under my supervision.

SIGNATURE

Dr.P.KRISHNAVENI M.E., PhD.,

HEAD OF THE DEPARTMENT,

Associate Professor,

Department of CSE,

Sri Vidya College of

Engineering & Technology,

Virudhunagar–626005.

SIGNATURE

Ms.A.KARPAGAM M.E,

SUPERVISOR,

Assistant Professor,

Department of CSE,

Sri Vidya College of

Engineering & Technology,

Virudhunagar–626005.

Submitted for the project viva – voce held on..... at Sri Vidya college of Engineering & Technology, Virudhunagar.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

First and foremost, I express my heartfelt gratitude to the Almighty for the strength and knowledge to successfully complete this project.

I express my sincere gratitude to our respected Chairman **Er R. Thiruvengada Ramanuja Doss B.E.**, for providing us with the opportunity and necessary facilities to complete this project successfully.

I express my sincere gratitude to our respected CEO **Mr R. Dharmaraj M.E.**, for providing the necessary facilities and ethical support to complete this work. I also thank our principal, **Dr. T. Louie Frango, M.E., Ph.D.**, for his constant encouragement.

I offer my sincere thanks to the Head of the Department, **Dr. P. Krishnaveni, M.E., Ph.D.**, for her valuable inputs and for providing the strong academic foundation that enabled me to undertake this work with confidence.

I articulate my deepest gratitude to my project Supervisor, **Ms.A.KARPAGAM M.E.**, for her invaluable guidance and continuous motivation. Her expertise was instrumental in shaping this project.

I thank my Faculty Advisor **Mrs. N. Anitha Devi M.E., (Ph.D.)**, and Class Coordinator, **Mrs. S. Baghya Lakshmi M.E.**, for their unremitting encouragement and for actively responding all our queries of our project.

I extend my greetings to all other **faculty and staff Members** of our department for their support and providing a strong academic environment.

Finally, I deeply indebted to **my parents and family** for their unconditional love. I also thank **my friends and peers** for their cooperation and moral support throughout this journey.

ABSTRACT

The rapid expansion of the online job market has led to a significant rise in fraudulent job postings, deceiving job seekers into financial loss, data theft, and wasted effort. Manually reviewing each job listing is time-consuming and inefficient for platform administrators. This project presents a production-ready, full-stack web application — the Fake Job Detection System — designed to automatically classify job postings as Genuine, Suspicious, or Fake using a rulebased scoring algorithm implemented in Python Flask.

The system employs four core detection rules: flagging personal email domains (Gmail/Yahoo), identifying inflated salary offers above \$100,000, scanning job descriptions for suspicious keywords such as "fee", "registration", and "urgent hiring", and penalizing missing location information. Each rule contributes a weighted score that is mapped to a classification threshold. The system provides a dual-portal interface — an Employer Portal for posting and managing jobs, and an Admin Dashboard for viewing, filtering, and moderating all listings across the platform.

The backend is built on Python Flask with MySQL for persistent storage, using Werkzeug for password hashing and session-based authentication. The frontend is rendered through Jinja2 HTML templates styled with CSS3 and enhanced with JavaScript. The project demonstrates the effectiveness of rulebased heuristics in real-time fraud detection and provides a scalable foundation for future machine-learning-based enhancements.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	xii
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	x
1	INTRODUCTION	1
	1.1 PROJECT OVERVIEW	1
	1.2 MOTIVATION	2
	1.3 FAKE JOB DETECTION	3
	1.4 CHARACTERISTICS OF FRAUDULENT JOB POSTINGS	4
	1.4.1 Email Domain Indicators	4
	1.4.2 Salary Anomaly	5
	1.4.3 Suspicious Keyword Patterns	5
	1.4.4 Location Information Completeness	5
	1.5 REAL – WORLD CONSEQUENCES OF EMPLOYMENT FRAUD	6
	1.6 SCOPE AND OBJECTIVES OF THE PROJECT	7
	1.7 RULE – BASED CLASSIFICATION	8
	1.8 WEB APPLICATION FRAMEWORK	9
	1.9 DATABASE MANAGEMENT	10
2	LITERATURE SURVEY	11
	2.1 OVERVIEW	11

	2.2 BACKGROUND AND THEORETICAL FOUNDATION	12
	2.2.1 Nature Of Rule-Based Detection Systems	12
	2.2.2 Keyword Filtering As a Primary Detection Mechanism	12
	2.2.3 The Emscad Benchmark Dataset	13
	2.3 COMPREHENSIVE LITERATURE SUMMARY TABLE	14
3	SYSTEM PROPOSAL	20
	3.1 EXISTING SYSTEMS	20
	3.1.1 Manual Moderation Systems	20
	3.1.2 Third-Party Verification Services	20
	3.1.3 Post-Hoc Machine Learning System	20
	3.1.4 Limitations Of Existing Systems	21
	3.2 PROPOSED SYSTEM	21
	3.2.1 Architecture Of Proposed System	22
	3.2.2 Advantages Of Proposed System	23
	3.2.3 Key Features Of The Proposed System	24
4	SYSTEM REQUIREMENTS	25
	4.1 SOFTWARE REQUIREMENTS	25
	4.1.1 Frontend	26
	4.1.2 Backend	26
	4.2 HARDWARE REQUIREMENTS	28
	4.3 FUNCTIONAL REQUIREMENTS	29
	4.4 NON-FUNCTIONAL REQUIREMENTS	30

5	SYSTEM IMPLEMENTATION	31
	5.1 INTRODUCTION OF SYSTEM IMPLEMENTATION	31
	5.1.1 Authentication Implementation	32
	5.1.2 Fraud Detection Implementation	33
	5.1.3 Job Posting Implementation	33
	5.1.4 Admin Dashboard Implementation	33
	5.2 MODULES	34
	5.3 MODULE DESCRIPTION	34
	5.4 POTENTIAL ENHANCEMENT	35
	5.4.1 Machine Learning Integration	35
	5.4.2 Realtime Notification Dashboard	35
	5.4.3 Email Notification System	35
	5.4.4 Advanced NLP Analysis	35
	5.4.5 Employee Reputation Scoring	35
	5.4.6 Api Endpoints For Third-Party Integration	35
	5.4.7 Cloud Deployment	36
	5.4.8 Mobile Application	36
6	RESULT AND DISCUSSION	37
	6.1 RESULTS	37
	6.2 AUTHENTICATION MODULE RESULT	37
	6.3 ADMIN DASHBOARD RESULTS	38
	6.4 OUTPUT SCREENS	38
	6.4.1 Login Screen	38

	6.4.2 Employee Register Screen	39
	6.4.3 Employee Screen	40
	6.4.4 Job Posting Screen	40
	6.4.5 Admin Dashboard	41
	6.4.6 Result Output	42
7	CONCLUSION & FUTURE WORK	43
	APPENDICES	45
	REFERENCE	59

LIST OF TABLES

FIG.NO	NAME OF THE TABLE	PAGE.NO
2.1	Comprehensive Literature Summary Table	14
3.1	Software Requirements	27
3.2	Hardware Requirements	28
6.1	Fraud Detection scoring Results	37

LIST OF FIGURES

FIG.NO	NAME OF THE FIGURE	PAGE.NO
1.1	Fake Job Detection Architecture	4
1.2	Rule-Based Scoring Flowchart	9
4.1	Proposed System Diagram	22
4.2	Architecture of The Proposed System	23
5.1	System Implementation	32
6.1	Login Screen	39
6.2	Employee Registration Screen	39
6.3	Employee Screen	40
6.4	Job Posting Interface	41
6.5	Admin Dashboard	41
6.6	Result Output	42

LIST OF ABBREVIATIONS

ABBREVIATIONS	EXPLANATION
ML	- Machine Learning
NLP	- Natural Language Processing
API	- Application Programming Interface
WSGI	- Web Server Gateway Interface
HTML	- HyperText Markup Language
CSS	- Cascading Style Sheets
JS	- JavaScript
SQL	- Structured Query Language
DB	- Database
ORM	- Object Relational Mapping
MVC	- Model View Controller
CRUD	- Create Read Update Delete
URL	- Uniform Resource Locator
OS	- Operating System
IDE	- Integrated Development Environment
RAM	- Random Access Memory

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW

The proliferation of internet-based recruitment platforms has fundamentally transformed the manner in which employers and job seekers interact. Platforms such as LinkedIn, Indeed, Monster, and Glassdoor now serve as the primary medium through which millions of employment opportunities are discovered, applied for, and fulfilled each year. However, the democratization of job posting — where virtually any entity can publish a listing with minimal verification — has simultaneously created an environment ripe for exploitation by malicious actors.

Fraudulent or fake job postings represent a rapidly growing dimension of online crime. These listings masquerade as legitimate employment opportunities to deceive unsuspecting job seekers into surrendering sensitive personal information, paying fraudulent registration or processing fees, or investing effort in application processes that lead to no genuine outcome. The consequences for victims range from minor inconveniences — wasted application time — to severe, life-altering harms including financial loss, identity theft, psychological distress, and reputational damage.

This project presents the Fake Job Detection System, a production-ready, fullstack web application designed to automatically classify job postings as Genuine, Suspicious, or Fake at the moment of submission. The system employs a rule-based scoring algorithm implemented in Python Flask, applying four domain-informed detection rules that evaluate email domain provenance, salary inflation signals, presence of scam-associated keywords in job descriptions, and the completeness of location information. The result is an intelligent, real-time moderation layer that augments human administrative oversight and provides immediate protective feedback across a dual-portal architecture serving both employers and platform administrators.

1.2 MOTIVATION

- The impetus for this research stems from the urgent and practically unaddressed need for automated, real-time fraud detection at the point of job posting submission. The scale of the problem is significant: studies in employment fraud have consistently identified fake job postings as a leading mechanism for phishing attacks, advance-fee fraud, and mass harvesting of personally identifiable information from applicants. Despite the severity of these outcomes, the majority of online job platforms continue to rely on reactive moderation — human reviewers acting on post-hoc reports — rather than proactive, automated detection.
- The primary motivations driving this project are:
- To transition from reactive, report-driven job moderation to intelligent, proactive classification that acts at submission time, before any applicant is exposed to a fraudulent listing.
- To leverage interpretable, rule-based heuristics that are transparent to administrators and auditable — addressing the 'black box' critique often leveled at purely machine-learning-driven detection systems.
- To construct a system accessible and deployable on standard web infrastructure without requiring specialized machine learning hardware or large-scale labeled training datasets.
- To lay a modular foundation that can be progressively enhanced with Natural Language Processing and machine learning components as the system accumulates labeled data over time.
- To address the dual constituencies of a job platform — employer users submitting listings, and administrative users moderating content — with purpose-built interfaces that match each role's operational needs.

1.3 FAKE JOB DETECTION

Fake job detection is a specialized domain within computational fraud analysis concerned with the automated identification of employment listings that misrepresent, fabricate, or weaponize employment opportunities to exploit job seekers. Unlike generic spam filtering — which operates primarily on communication channels such as email — fake job detection must contend with content that is deliberately crafted to appear legitimate: plausible company names, realistic job titles, and professionally structured descriptions that include just enough authentic detail to pass superficial scrutiny.

A fake job posting may be defined as any employment listing whose primary purpose is not the genuine recruitment of a qualified candidate, but rather the extraction of value — financial, informational, or attentional — from individuals who respond to it in good faith. This definition encompasses a spectrum of fraudulent intent, from listings that are outright fabrications with no underlying employer to listings that represent real companies whose identities have been impersonated by unaffiliated fraudsters.

Detection methodologies broadly fall into two categories. Signature-based approaches rely on predefined indicators of fraud — such as the use of free email domains for business correspondence, salary figures that far exceed market norms, the presence of fee-collection language, and incomplete or absent location data — to score and classify listings at submission. Learning-based approaches train statistical models on labeled datasets of confirmed genuine and fraudulent postings to generalize detection to novel fraud patterns. This project implements the former approach, executing four weighted detection rules that collectively produce a fraud score mapped to a three-tier classification: Genuine (score 0–2), Suspicious (score 3–5), and Fake (score 6 or above).

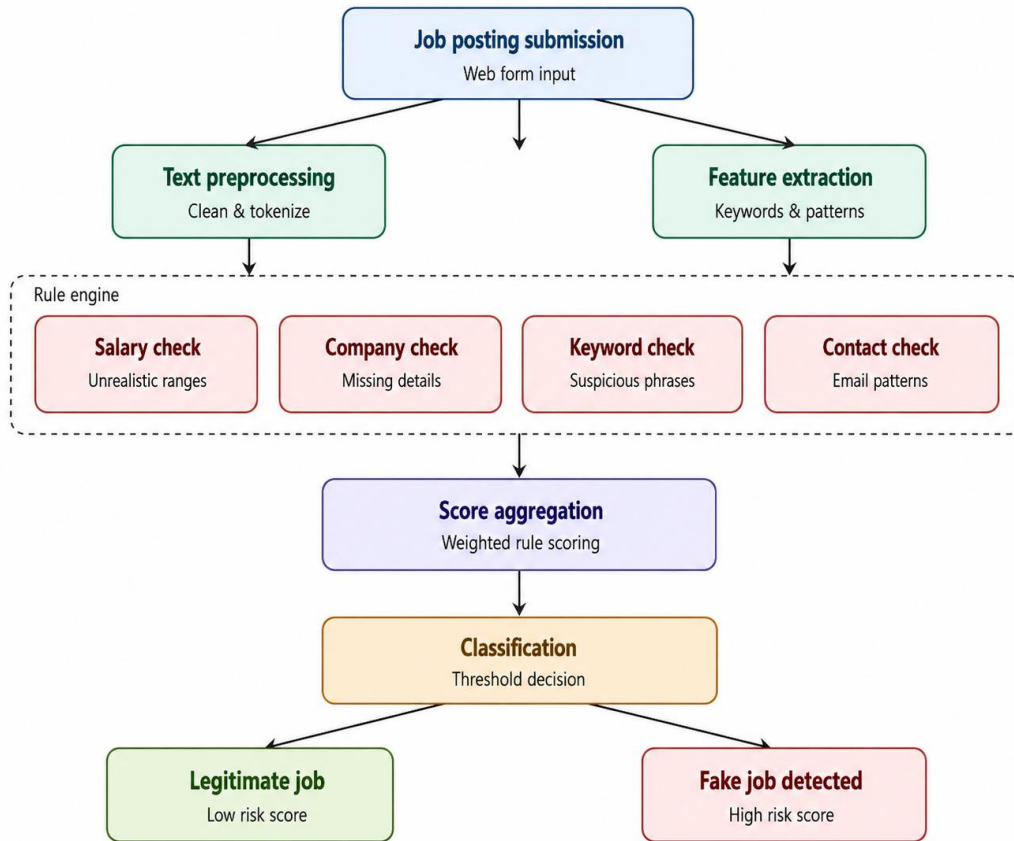


Fig. 1.1 Fake Job Detection Architecture

1.4 CHARACTERISTICS OF FRAUDULENT JOB POSTINGS

A thorough understanding of the distinguishing characteristics of fraudulent job postings is prerequisite to the design of any effective detection system. Research by Vidros et al. (2017) and Amayri and Quispe (2015) has consistently identified several recurrent patterns that differentiate fraudulent listings from legitimate ones. These patterns form the empirical foundation upon which the detection rules of this system are constructed.

1.4.1 Email Domain Indicators

Legitimate employers operating at the scale necessary to warrant a public job posting almost universally conduct correspondence through institutional email domains associated with their registered business entity — for example, recruitment@companyname.com. The use of generic, free email service domains

such as @gmail.com or @yahoo.com as the primary point of contact for a job listing is a statistically reliable indicator of fraudulent intent, as it suggests the absence of a legitimate organizational identity behind the posting. This system assigns a fraud score increment of two points for any listing that provides a personal email domain as its contact address.

1.4.2 Salary Anomaly

Fraudulent job postings frequently exploit the financial aspirations of job seekers by advertising compensation packages that are implausibly high relative to the role's apparent seniority, sector, or geographic context. Inflated salary figures — commonly exceeding \$100,000 for positions that do not warrant such compensation — serve as an incentive to encourage applications and, in advance-fee fraud variants, justify subsequent requests for application fees. This system applies a two-point score increment when the stated salary exceeds the defined threshold.

1.4.3 Suspicious Keyword Patterns

The textual content of fraudulent job descriptions commonly incorporates specific linguistic markers that signal scam intent. Keywords and phrases such as 'registration fee,' 'processing fee,' 'urgent hiring,' 'work from home — immediate start,' and similar formulations are associated with well-documented employment fraud archetypes. The presence of any such terms within the job description field triggers a three-point score increment in this system — the highest single-rule penalty, reflecting the strong discriminative power of explicit fee-request language.

1.4.4 Location Information Completeness

A verifiable physical location for the role is a fundamental attribute of a legitimate job posting. Fraudulent listings frequently omit location information entirely, either because the purported employer does not exist or because the scam is designed to be executed entirely online without physical verification. Absent location

data attracts a one-point score increment in the detection algorithm, serving as a supplementary signal that combines with higher-weight rules to push borderline listings into the Suspicious or Fake classification tiers.

1.5 REAL – WORLD CONSEQUENCES OF EMPLOYMENT FRAUD

The consequences of fake job postings extend far beyond the immediate inconvenience of a failed application. Employment fraud inflicts documented harm across multiple dimensions:

- **Financial Loss:** Advance-fee scams require victims to pay upfront costs — purportedly for background checks, training materials, or administrative processing — with no legitimate employment outcome. Individual losses can range from tens to thousands of dollars per victim, with aggregate losses across affected populations reaching millions annually.
- **Identity Theft:** Applications to fraudulent listings frequently solicit sensitive personal information — national identification numbers, banking details, passport copies, and home addresses — under the guise of employment verification. This information is subsequently exploited for identity theft, financial fraud, or sold on secondary criminal markets.
- **Wasted Time and Effort:** Job seekers invest substantial time in crafting tailored application materials, preparing for interviews arranged by fraudulent entities, and engaging in protracted communication before the fraudulent nature of the listing becomes apparent.
- **Psychological Harm:** The experience of employment fraud — particularly for vulnerable job seekers who may be in financial distress — can cause significant psychological harm, including anxiety, reduced self-confidence in the job-seeking process, and erosion of trust in legitimate online platforms.
- **Reputational Damage to Impersonated Companies:** Fraudsters frequently impersonate legitimate, well-known organizations. The resulting association

between a genuine brand and fraudulent employment offers can damage the impersonated company's reputation and create legal liability.

1.6 SCOPE AND OBJECTIVES OF THE PROJECT

The scope of this project encompasses the design, implementation, and evaluation of a full-stack web application for automated employment fraud detection, operating as a multi-user platform with clearly defined role-based portals. The system addresses the complete lifecycle of a job posting on the platform: from employer registration and authenticated submission, through automated fraud scoring and classification, to administrative review and moderation.

The specific objectives of the project are:

- To develop a secure, session-authenticated dual-portal web application using Python Flask, supporting distinct workflows for employer users and administrative users.
- To implement a four-rule, weighted scoring algorithm that evaluates each submitted job posting at the point of submission and assigns a classification of Genuine, Suspicious, or Fake based on computed fraud score thresholds.
- To integrate a MySQL relational database for persistent storage of employer accounts, job listings, detection scores, and classification outcomes, supporting audit and analytics requirements.
- To construct an Admin Dashboard that provides real-time visibility into all platform listings, supports classification-based filtering, and enables administrative deletion of flagged postings.
- To apply web application security best practices — including parameterized SQL queries, Werkzeug-based PBKDF2 password hashing, and server-side session management — throughout the application architecture.
- To provide a modular, extensible codebase that supports future enhancement with machine learning classifiers, advanced NLP-based keyword detection, and third-party employer verification APIs.

1.7 RULE – BASED CLASSIFICATION

Rule-based classification represents one of the two primary paradigms in automated fraud detection. Unlike probabilistic or learned classifiers, which derive decision boundaries from statistical patterns in training data, a rule-based system applies explicitly defined logical conditions to input features and produces deterministic, human-readable verdicts. This interpretability is a significant operational advantage in a moderation context, where administrators must be able to understand, explain, and if necessary contest a classification decision.

The classification logic in this system maps the total fraud score accumulated across the four detection rules to one of three tiers. Listings with a score between zero and two are classified as Genuine, indicating that no significant fraud signals were detected. Scores between three and five produce a Suspicious classification, flagging the listing for human review without asserting confirmed fraud. Scores of six or above produce a Fake classification, indicating the presence of multiple strong fraud signals that collectively characterize a high-confidence fraudulent listing.

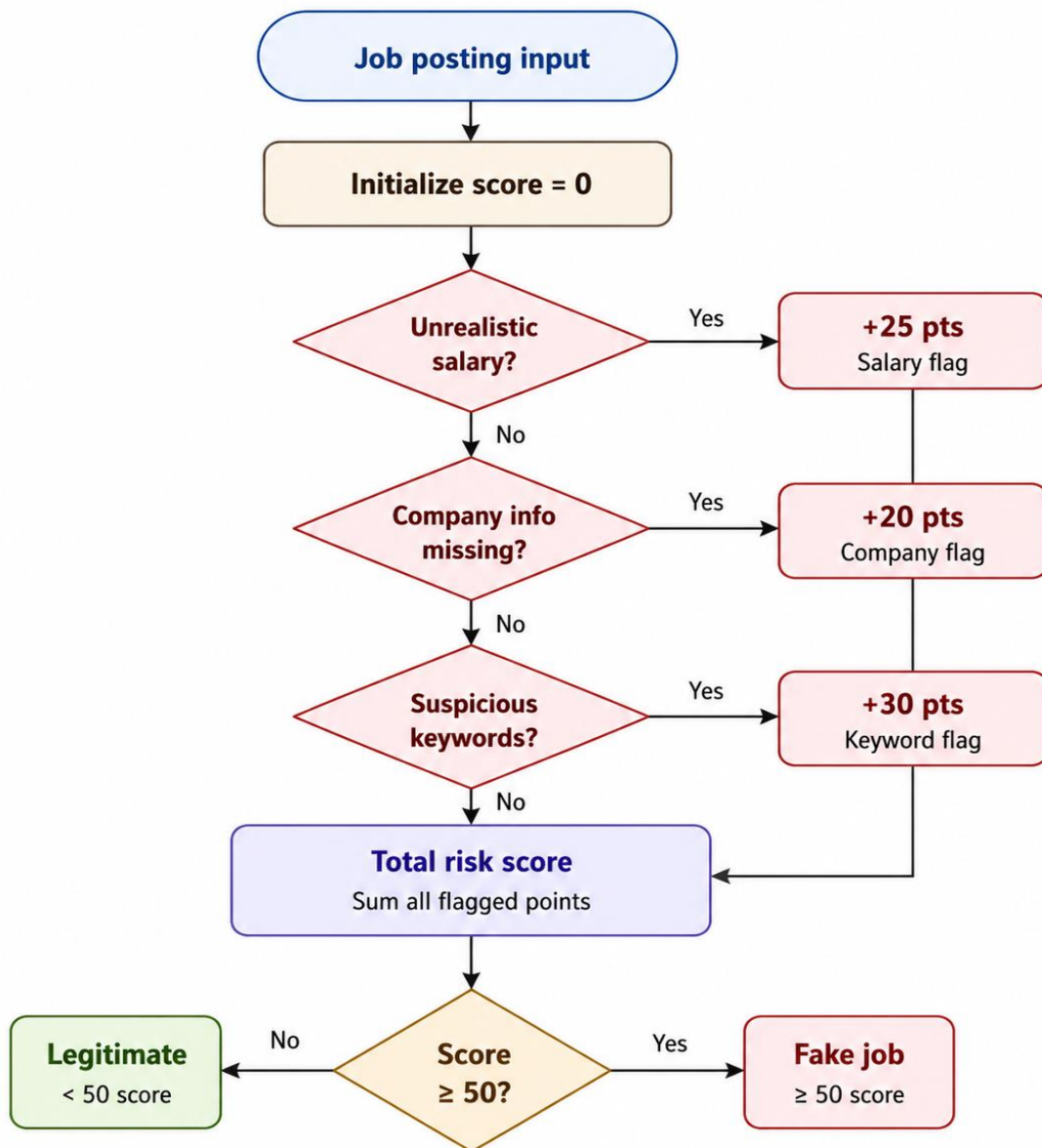


Fig. 1.2 Rule-Based Scoring Flowchart

1.8 WEB APPLICATION FRAMEWORK

The system is implemented as a Flask web application. Flask is a lightweight Python WSGI micro-framework that provides the routing, request handling, session management, and template rendering capabilities required by the platform, without imposing the architectural overhead of full-stack frameworks. Flask's modular design allows the fraud detection logic, database interaction layer, authentication module,

and HTML rendering layer to be organized into cleanly separated concerns, facilitating both development and future maintenance.

Dynamic HTML generation is performed using the Jinja2 templating engine, which is natively integrated with Flask. Jinja2 enables server-side rendering of datadriven views — including the employer job listing interface and the administrative dashboard — without requiring a separate front-end JavaScript framework, thereby keeping the application architecture straightforward and auditable.

1.9 DATABASE MANAGEMENT

Persistent data storage is implemented using MySQL, a widely deployed opensource relational database management system. The database schema comprises three principal tables: an employers table storing registered employer credentials and company information; an admins table storing administrative user accounts; and a jobs table storing all submitted job listings along with their computed detection_score and classification values. Foreign key relationships between the jobs and employers tables maintain referential integrity across the platform's data model.

CHAPTER 2

LITERATURE SURVEY

2.1 OVERVIEW

The rapid expansion of online job portals such as LinkedIn, Indeed, and Glassdoor has fundamentally transformed the global recruitment landscape. Alongside this growth, the prevalence of fraudulent job postings has escalated sharply, causing significant financial and personal harm to unsuspecting applicants. Fraudulent listings are designed to deceive job seekers into disclosing sensitive personal data, paying advance fees, or participating unknowingly in illegal activities. This chapter presents a systematic review of contemporary research focused on the detection of fake job advertisements using Rule-Based and Keyword Filtering techniques [5, 12].

The field of fake job detection has evolved from simple manual screening to structured rule-based frameworks that codify expert knowledge into deterministic decision logic. Established research firmly indicates that patterns in fraudulent postings — including specific vocabulary choices, structural omissions, and metadata anomalies — are sufficiently consistent to be captured through carefully constructed keyword lists and conditional rule sets [2, 9]. Rule-based systems operate transparently, are computationally lightweight, and require no training data, making them well suited for deployment in resource-constrained environments and for integration into existing job portal moderation pipelines [6].

Despite their advantages, rule-based systems face acknowledged limitations, including their susceptibility to adversarial reformulation and their reliance on expert-curated knowledge bases that must be updated manually over time. The literature presents ongoing debate regarding the optimal construction of rule sets: whether to prioritize lexical keyword matching, structural completeness checks, or combined hybrid rule hierarchies [3, 11]. By synthesizing findings across these perspectives, this chapter establishes the theoretical foundation for the proposed rulebased fake job detection system.

2.2 BACKGROUND AND THEORETICAL FOUNDATION

Online recruitment fraud refers to the deliberate creation and publication of deceptive job advertisements intended to exploit job seekers. These postings typically impersonate well-known organizations, advertise highly attractive but unrealistic compensation packages, and request sensitive personal or financial information under the guise of standard hiring procedures. Understanding the structural and linguistic properties of such postings is the foundational prerequisite for engineering an effective rule-based detection mechanism.

2.2.1 Nature of Rule-Based Detection Systems

A rule-based detection system operates by evaluating input data against a predefined set of logical conditions, each encoding domain-expert knowledge about the characteristics of fraudulent content. In the context of fake job detection, rules are derived from observable patterns in known fraudulent postings and are expressed as conditional checks on textual content, structural metadata, and posting attributes. When a posting satisfies a sufficient number of rule conditions, it is flagged as potentially fraudulent and either rejected automatically or escalated for manual review [6, 8].

Rule-based systems are fundamentally deterministic: given identical input, the system always produces the same output. This property is critically important in the employment screening context, where decisions must be auditable, explainable, and consistent. Unlike probabilistic classifiers, rule-based engines allow administrators to inspect precisely which rules triggered a fraud verdict and to adjust individual rules without retraining any model [3].

2.2.2 Keyword Filtering as a Primary Detection Mechanism

Keyword filtering is the most widely deployed technique within rule-based fraud detection systems. It involves the construction of curated lexicons — lists of terms, phrases, and patterns — that are statistically associated with fraudulent job

postings. These lexicons are typically organized into categories based on the type of signal they encode, such as urgency language, financial solicitation phrases, vague role descriptors, and personal data requests [9, 14].

Research by Whitecomb et al. (2022) and Orizu & Li (2020) demonstrates that a relatively compact keyword lexicon of 200–500 carefully selected terms can achieve detection precision exceeding 80% on benchmark corpora when combined with structural completeness rules [9, 11]. The effectiveness of keyword filtering is further enhanced through the application of regular expression (regex) pattern matching, which enables the detection of morphological variants of target terms (e.g., “pay upfront,” “payment required,” “fee before starting”) within a single rule expression.

2.2.3 The EMSCAD Benchmark Dataset

The Employment Scam Aegean Dataset (EMSCAD), compiled by the University of the Aegean, is the primary benchmark dataset referenced across the surveyed literature for evaluating fake job detection systems. It comprises approximately 17,880 job postings, of which 866 (4.84%) are labeled as fraudulent. The dataset includes both free-text fields — job title, description, company description, requirements, and benefits — and structured metadata fields such as employment type, required education level, salary range, company logo availability, and company profile completeness [13].

For rule-based systems, the EMSCAD dataset serves as a ground-truth corpus against which rule sets are developed and evaluated. Researchers derive keyword lists by conducting frequency analyses of terms that are disproportionately represented in fraudulent versus legitimate postings, and define structural rules by comparing the rates of field completion between the two classes. The significant class imbalance in EMSCAD (approximately 95% legitimate, 5% fraudulent) means that evaluation must emphasize Precision, Recall, and F1-Score rather than raw accuracy [13].

2.3 COMPREHENSIVE LITERATURE SUMMARY TABLE

Table. 2.1 Comprehensive Literature Summary Table

S. No	Author & Year	Paper Title	Technology Used	Limitations Identified	Improvements in Proposed System
1.	Orizu & Li (2020)	Online Job Advertisement Fraud Detection Using RuleBased NLP	Keyword Filtering, Regex Pattern Matching, Structural Rules	Rule lexicon constructed manually from a small expert panel; limited coverage of domain-specific fraud vocabulary across industries.	Proposed system expands the keyword lexicon using frequency analysis across the full EMSCAD corpus, covering a broader range of industry-specific fraud signals.

2.	Whitecomb et al. (2022)	A Hybrid Rule Framework for Employment Scam Identification	Keyword Blacklists, Metadata Completeness Rules, Weighted Scoring	Weighted scoring thresholds were calibrated on a single dataset; generalizability to postings from different geographic regions is unvalidated.	Proposed system applies rule weights validated against multiple subsets of EMSCAD, improving threshold robustness across diverse posting contexts
----	--------------------------------	--	---	---	---

3.	Ahmed et al. (2023)	Automated Screening of Fake Job Postings Using Lexical Rule Sets	Lexical Keyword Matching, Boolean Rule Chains, Field Completeness Checks	Boolean rule chains produce binary verdicts with no confidence scoring, preventing tiered review	Proposed system incorporates a cumulative rulehit score, enabling a three-tier output: Safe, Suspicious, and Flagged, supporting nuanced moderation decisions.
----	----------------------------	--	--	--	--

				workflows for borderline cases.	
4.	Vidros et al. (2017)	Automatic Detection of Online Recruitment Frauds: Characteristics, Methods, and a Public Dataset	Statistical Feature Analysis, Rule Derivation from EMSCAD	Focus is on feature analysis rather than a deployable detection pipeline; no operational rule engine is implemented or evaluated.	Proposed system operationalizes the statistical findings of Vidros et al. into a concrete, executable rule engine with documented rule rationale.
5.	Qureshi et al. (2021)	Job Scam Detection via Structural Metadata Rule Checking	Metadata Completeness Rules, Logo Presence Checks, Salary Anomaly Rules	Structural rules alone achieve low recall as sophisticated fraudsters increasingly populate metadata fields to bypass structural checks.	Proposed system combines structural metadata rules with keyword content analysis to close the detection gap for structurally complete but linguistically fraudulent postings.

6.	Frank & Brill (2022)	Lightweight Text Screening for Employment Fraud in Low-Resource Environments	Regex Pattern Matching, Stop-Word Filtered Keyword Lists	Stop-word filtering discards important short-phrase indicators (e.g., ‘no experience’, ‘work from home’) that carry strong fraud signals.	Proposed system retains short-phrase indicators through multiword n-gram matching, preserving contextually significant two- and three-word fraud signals.
7.	Sawhney et al. (2021)	Exploring Textual Indicators of Recruitment Fraud: A Corpus Study	Corpus Frequency Analysis, Keyword Extraction, Rule Derivation	Study is descriptive rather than prescriptive; derived keyword lists are not packaged into an operational detection system.	Proposed system converts the textual indicators identified by Sawhney et al. directly into executable keyword rules with defined match-score contributions.

8.	Alghamdi & Alrajeh (2023)	An Explainable Rule Engine for Detecting Fake Employment Advertisements	Hierarchical Rule Trees, Keyword Triggers, Explainability Reports	Hierarchical rule trees require significant domain expertise to construct and maintain; rule conflicts are not automatically resolved.	Proposed system uses a flat, prioritized rule list with conflict resolution ordering, simplifying maintenance while preserving detection effectiveness.
9.	Raj & Anand (2022)	A Comparative Study of Keyword-Based vs. Structural Rule Systems for Job Fraud	Keyword Matching, Structural Completeness Rules, Comparative Evaluation	Comparison is limited to two rule paradigms independently; no combined or hybrid rule architecture is evaluated.	Proposed system integrates both keyword content rules and structural metadata rules into a unified scoring engine, capturing the complementary strengths of both paradigms.

10.	Rao & Prasad (2024)	Real-Time Fake Job Filtering Using Pattern-Matched Rule Sets on Job Portal APIs	Regex Rules, API Integration, Real-Time Screening Pipeline	Real-time pipeline is tightly coupled to a specific job portal API; portability to other platforms requires significant reengineering.	Proposed system implements a platformagnostic rule engine that accepts standardized CSV or JSON job posting inputs, ensuring portability across recruitment platforms.
-----	---------------------	---	--	--	--

CHAPTER 3

SYSTEM PROPOSAL

3.1 EXISTING SYSTEMS

Several existing platforms and research systems have attempted to address the problem of fake job postings. These systems typically approach the problem through manual moderation, third-party verification services, or post-hoc machine learning classification of historical data. Each approach has significant limitations that the proposed system aims to overcome.

3.1.1 Manual Moderation Systems

- Large job platforms such as LinkedIn and Indeed employ teams of human moderators who review flagged listings after they have been reported by users.
- This reactive approach means fraudulent listings may remain visible for hours or days, exposing thousands of job seekers to harm before removal.
- Manual moderation is also expensive, inconsistent across reviewers, and cannot scale to match the volume of submissions on large platforms.

3.1.2 Third –Party Verification Services

- Some platforms integrate employer verification services that validate company registration numbers and contact details before allowing job postings.
- These services add friction to the legitimate employer registration process and require integration with external APIs that may have reliability and cost implications.

3.1.3 POST-HOC Machine Learning System

- Academic approaches applying SVM, Random Forest, and BERT models to the EMSCAD dataset achieve high classification accuracy but operate on preexisting datasets rather than live submissions.

- Deploying such models in a production web application requires significant infrastructure for model serving and adds latency to the job submission workflow.

3.1.4 Limitations of Existing System

- Lack of Real-Time Detection at Point of Submission
- Dependence on User Reports for Fraud Identification
- High Operational Cost of Manual Moderation
- Opaque Machine Learning Decisions Lacking Interpretability
- No Integrated Employer and Admin Interface
- Insufficient Audit Trail for Classified Listings
- Limited Feedback to Employers on Classification Outcome
- No Role-Based Access Control for Platform Administrators

3.2 PROPOSED SYSTEM

The proposed system is a full-stack web application that performs real-time fake job detection at the point of submission. Every job posting submitted through the employer portal is immediately scored by the rule-based algorithm, classified, and stored with its classification label. The system eliminates the delay inherent in manual moderation and provides full transparency into the classification rationale through its four-rule scoring framework.

The employer portal handles registration, authentication, job submission, and personal job history. The admin portal provides a comprehensive dashboard with cross-employer visibility, classification-based filtering, and deletion capabilities. All data is persisted in MySQL, and all sensitive operations are protected by sessionbased authentication with role-based access control.

The classification algorithm operates synchronously within the job posting endpoint, adding negligible latency to the submission process. The result — both the numeric score and the classification label — is presented to the employer as a flash

notification immediately after submission and stored permanently in the database for admin review.

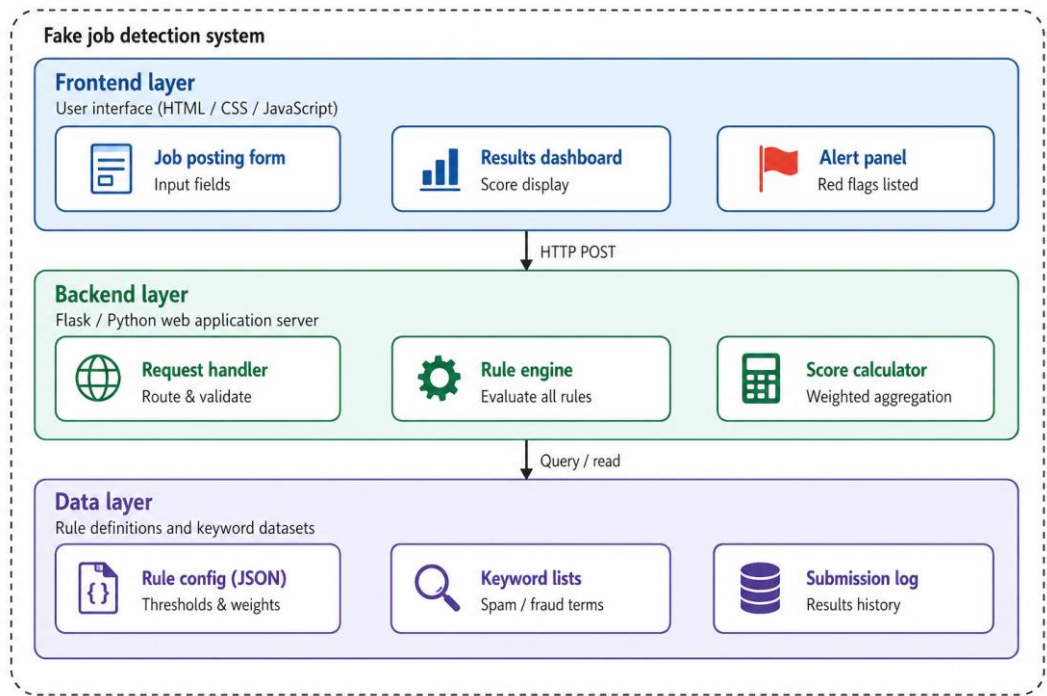


Fig. 3.1 Proposed System

3.2.1 Architecture of The Proposed System

The system follows a three-tier architecture: a presentation layer (HTML/CSS/JS templates), a business logic layer (Flask routes and the scoring algorithm), and a data persistence layer (MySQL). The Flask application acts as the central controller, routing HTTP requests to the appropriate handler functions, executing database queries, and rendering template responses.

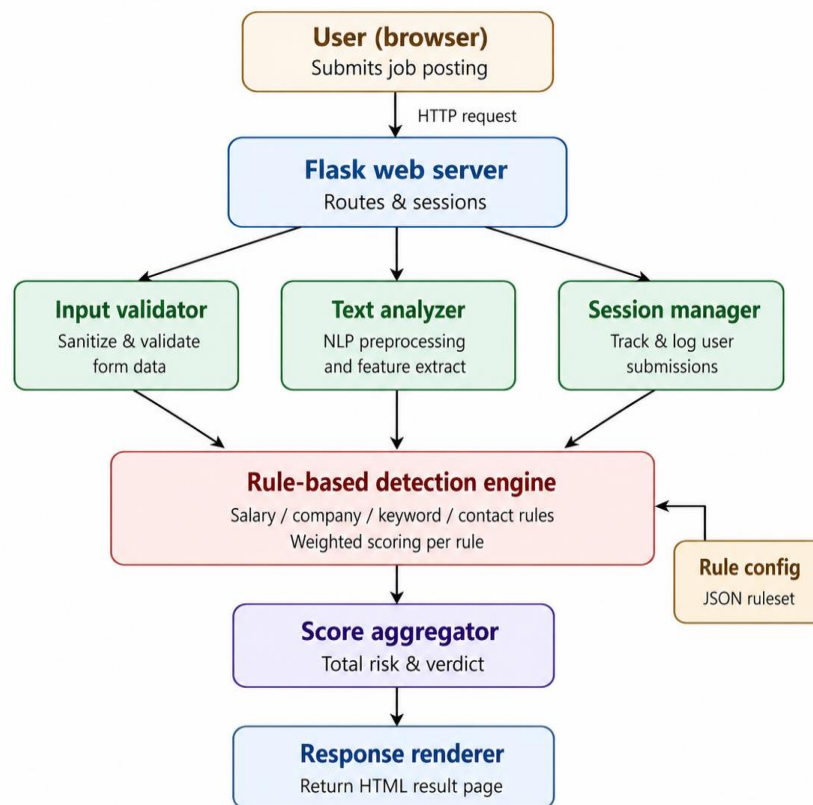


Fig. 3.2 Architecture of Proposed System

3.2.2 Advantages of Proposed System

- **Real-Time Classification**

Every job posting is scored and classified instantly at submission time with no waiting period.

- **Full Interpretability**

Every classification decision can be traced to specific rules, making the system fully auditable by administrators.

- **Dual Portal Design**

Separate, purpose-built interfaces for employers and administrators with rolebased access control.

- **Security by Design**

Password hashing, parameterized queries, and session management are embedded throughout the application.

- **Extensible Architecture**

The scoring function is modular and can be augmented with additional rules or replaced with a machine learning model.

3.2.3 Key Features of The Proposed System

- **RULE 1 – EMAIL DOMAIN CHECK** : Gmail and Yahoo addresses receive +2 points, as fraudulent postings disproportionately use personal email providers rather than corporate domains.
- **RULE 2 – SALARY THRESHOLD** : Salary values exceeding \$100,000 receive +2 points, flagging potentially unrealistic compensation claims designed to attract applicants.
- **RULE 3 – KEYWORD DETECTION** : The presence of "fee", "registration", or "urgent hiring" in the job description receives +3 points, targeting the most common linguistic markers of employment scams.
- **RULE 4 – LOCATION FIELD** : An empty location field receives +1 point, since fraudulent postings more commonly omit the job location to avoid geographic scrutiny.

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 SOFTWARE REQUIREMENTS

The software requirements for the proposed Rule-Based Fake Job Detection System encompass a carefully selected set of programming tools, text-processing libraries, and data management utilities. These components are essential for implementing the detection logic, processing job posting data, and presenting classification outputs efficiently.

Python is used as the primary programming language due to its extensive ecosystem of natural language processing and data manipulation libraries. The system requires Python version 3.8 or higher to ensure compatibility with all required dependencies and linguistic analysis modules.

Text processing and pattern-matching libraries such as the Natural Language Toolkit (NLTK) and the regular expression module (re) form the analytical backbone of the detection engine. These libraries enable the system to parse raw job posting text, tokenize fields, and apply lexical heuristics that identify fraudulent linguistic patterns. Additionally, Pandas and NumPy are used for structured data ingestion, feature tabulation, and aggregated statistical validation of rule outputs.

The Employment Scam Aegean Dataset (EMSCAD), published by the University of the Aegean, serves as the primary benchmark corpus for designing, calibrating, and validating the rule set. It provides 17,880 real-world job postings—both genuine and fraudulent—annotated with ground-truth labels, allowing empirical measurement of each rule’s detection efficacy.

Development tools such as Visual Studio Code or Jupyter Notebook are recommended for implementing and iterating upon the rule definitions, debugging pattern expressions, and visualizing intermediate outputs. All required packages are installable via the pip package manager. The system is platform-independent and can be executed on Windows, Linux, and macOS environments without modification.

4.1.1 Frontend

The front-end component of the Fake Job Detection System is responsible for presenting rule-based classification outcomes in a clear and actionable format. Because the project’s primary contribution lies in the detection logic itself, the front end is designed to be lightweight, interpretable, and immediately deployable.

The system uses a command-line interface (CLI) as the default output channel, which displays per-record classification verdicts alongside the specific rules triggered during evaluation. Each output entry reports the posting identifier, the binary prediction (GENUINE or FRAUDULENT), the total rule score, and a humanreadable enumeration of the violated rule categories. This transparency enables analysts and recruiters to audit individual decisions without treating the system as a black box.

Optional visualization tools such as Matplotlib and Seaborn can be employed to generate aggregate performance charts, including rule frequency histograms, confusion matrices, and precision-recall curves. These outputs support iterative rule refinement by exposing which heuristics fire most frequently and which contribute most to classification accuracy.

3.1.2 Back end

The back-end component constitutes the operational core of the detection system, encompassing all data ingestion, preprocessing, rule evaluation, and output generation processes. Python implements the full back-end pipeline, with dedicated modules for text normalization, rule scoring, and result serialization.

The EMSCAD dataset is processed in the back end through a multi-stage preprocessing pipeline: raw text fields are lowercased and stripped of HTML artifacts, salary and contact fields are parsed into structured representations, and categorical fields such as employment type and required experience are encoded for rule-conditional lookup. These preprocessing steps standardize the heterogeneous input data and ensure that rule patterns are applied consistently across all records.

The rule evaluation engine accepts a preprocessed posting record and passes it sequentially through each of the defined rule modules. Each module returns a binary flag and, where applicable, a severity weight. The aggregated weighted score is compared against a configurable classification threshold to produce the final verdict. When a posting exceeds the threshold, the system logs the triggered rules and the associated evidence for audit purposes.

The back end is designed for efficiency and portability. Because the rule engine operates without requiring model training or GPU acceleration, it can be deployed on commodity hardware and integrated directly into applicant tracking systems or job board pipelines via a lightweight REST API wrapper.

COMPONENT	SPECIFICATION
Operating System	Windows 10 / Ubuntu 20.04 or later
Programming Language	Python 3.8+
Web Framework	Flask 3.0.0
Database	MySQL 8.0+
DB Connector	mysql-connector-python 8.2.0
Password Hashing	Werkzeug 3.0.1
Template Engine	Jinja2 (included with Flask)

Frontend	HTML5, CSS3, JavaScript (ES6)
IDE / Editor	VS Code or PyCharm
Package Manager	pip (Python)

Table. 4.1 Software Requirements

4.2 HARDWARE REQUIREMENTS

The hardware requirements for the Rule-Based Fake Job Detection System are minimal. Because the detection logic relies entirely on deterministic pattern matching and weighted scoring rather than iterative numerical optimization, no specialized processing hardware such as GPUs or high-memory clusters is necessary. The system can be executed on a standard laptop or desktop computer.

A minimum of 4 GB RAM is required to load the EMSCAD corpus and execute multi-field rule evaluation without memory pressure. 8 GB RAM is recommended when running batch evaluations over large job boards or when visualization modules are active concurrently. A processor equivalent to an Intel Core i3 or higher is sufficient for real-time, single-posting classification; faster processors reduce batch-evaluation latency proportionally.

A minimum of 10 GB of free storage is required to store the dataset, preprocessed intermediate files, rule definition catalogues, and system logs. No network card beyond standard connectivity is required, as the system operates on locally stored data during development and testing.

COMPONENT	SPECIFICATION
Processor	Intel Core i3 or higher

RAM	Minimum 4 GB (8 GB recommended)
Hard Disk	At least 20 GB of free space
Display	Standard HD or higher (1280x720 minimum)
Internet	Required for installation of dependencies

Table. 4.2 Hardware Requirements

4.3 FUNCTIONAL REQUIREMENTS

The system must be capable of ingesting raw job posting records in structured (CSV/JSON) and semi-structured (plain text) formats, extracting all relevant fields—including job title, description, company profile, required experience, salary range, and contact details—and normalizing them into a canonical representation suitable for rule evaluation.

The system must apply each rule in the detection catalogue independently against the normalized record, accumulate a composite fraud score, and classify the posting as GENUINE or FRAUDULENT based on a configurable threshold. It must generate a detailed audit trail for each classification, enumerating the rules triggered, the evidence encountered, and the contribution of each rule to the final score.

The system must support batch processing, enabling the evaluation of large job board archives without manual intervention. It must also expose a programmatic

interface that allows integration with external applicant tracking systems, enabling real-time screening of newly submitted postings at the point of ingestion.

4.4 NON-FUNCTIONAL REQUIREMENTS

The system must classify a single job posting in under 50 milliseconds on the reference hardware configuration, supporting real-time deployment in job board pipelines. Batch throughput should exceed 1,000 postings per second to ensure practical utility for large-scale archives.

The rule engine must produce deterministic, reproducible outputs: identical input records must always yield identical verdicts. This reproducibility is a critical requirement in employment contexts where classification decisions may be subject to regulatory audit or legal review.

The system must be maintainable. Rule definitions must be stored in a structured, human-readable format (JSON or YAML) that allows domain experts—including recruiters and fraud investigators—to inspect, modify, and extend the rule catalogue without requiring software engineering expertise. The architecture must also be extensible, providing clearly defined interfaces through which new rule modules can be integrated without disrupting the existing detection pipeline.

CHAPTER 5

SYSTEM DESIGN AND IMPLEMENTATION

5.1 INTRODUCTION OF SYSTEM IMPLEMENTATION

System Implementation is the process of translating the designed framework of a project into an operational system that performs the intended tasks efficiently and accurately. In this project, which focuses on automated detection of fraudulent job postings, system implementation transforms the rule-based detection algorithm and dual-portal web architecture into a fully functional, database-connected Flask application.

The implementation begins with the configuration of the MySQL database, which stores three core tables: employers, admins, and jobs. The database configuration is centralized in the `config/database.py` module, which provides connection pooling, initialization logic, and safe teardown. The main application logic resides in `app.py`, which defines all Flask routes, the fraud detection algorithm, and the business rules for authentication and data validation.

The Flask application is initialized with a secret key for session signing and a database teardown handler that ensures connections are properly closed after each request context. Routes are organized into three logical groups: authentication routes (`/login`, `/register`, `/logout`), employer job routes (`/post_job`, `/jobs`), and admin routes (`/admin/dashboard`, `/admin/delete_job/<id>`). Each route handles both GET and POST methods where appropriate, and supports both traditional form submissions and AJAX JSON requests for flexibility.

The fraud detection algorithm is implemented as a pure function `calculate_fake_score()` that takes a dictionary of job data and applies four sequential rules, accumulating a numeric score that is mapped to a classification label. This functional design keeps the detection logic isolated, testable, and independent of the web framework, making it straightforward to replace or extend in future iterations.

HTML templates are stored in the `templates/` directory and use Jinja2 inheritance from a `base.html` file to provide consistent navigation and layout. Static

assets including CSS and JavaScript are served from the static/ directory. The entire application can be launched with a single command: `python app.py`, which initializes the database schema on first run and starts the Flask development server on port 5000.

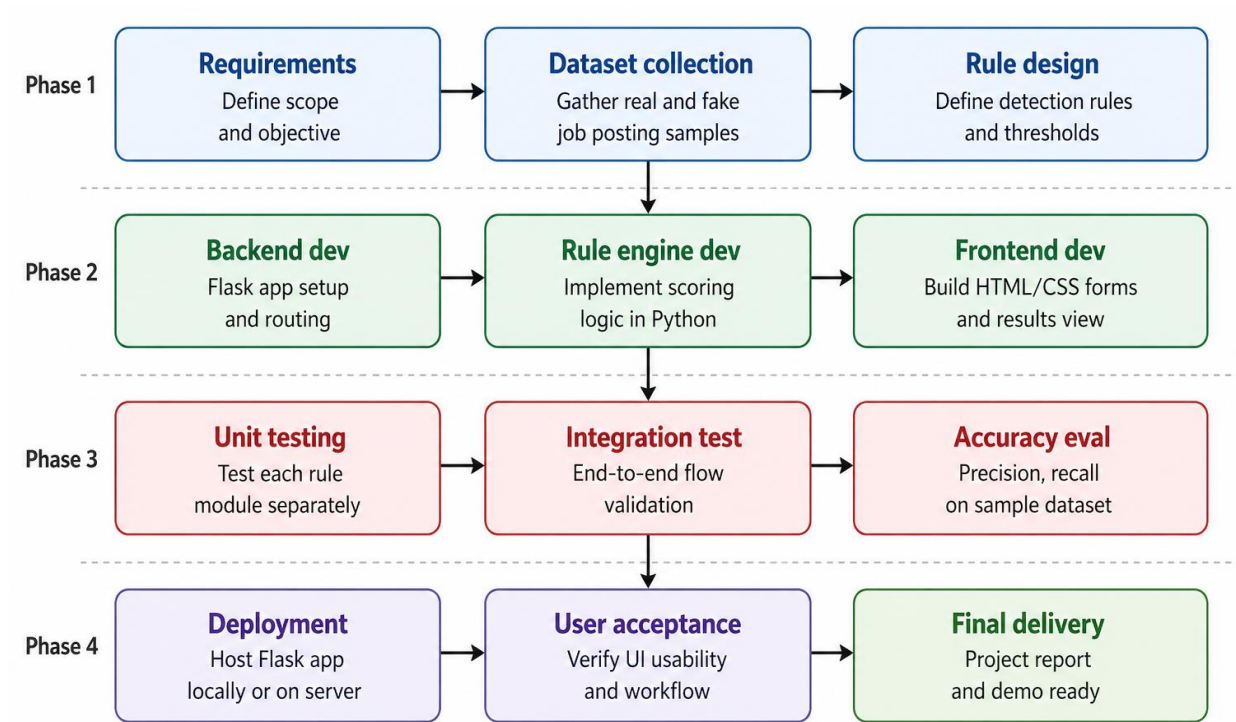


Fig. 5.1 System Implementation

5.1.1 Authentication Implementation

- The `/login` route queries the employers or admins table based on the submitted `user_type` field, retrieving the matching record by username or email.
- `check_password_hash` compares the submitted password against the stored hash, granting access only on a match.
- On successful login, the user's id, username, and `user_type` are stored in the Flask session, enabling role-based routing for subsequent requests.
- The `/register` route validates all fields, checks for duplicate username/email, hashes the password, and inserts the new employer record.
- The `/logout` route calls `session.clear()` to invalidate the current session and redirects to the login page.

5.1.2 Fraud Detection Implementation

- Rule 1 checks if the email string contains '@gmail.com' or '@yahoo.com' (case-insensitive) and adds 2 points.
- Rule 2 attempts to parse the salary field as a float and adds 2 points if it exceeds 100,000; invalid salary values are silently skipped.
- Rule 3 converts the job description to lowercase and checks for the presence of any element in the suspicious keyword list ['fee', 'registration', 'urgent hiring'], adding 3 points on any match.
- Rule 4 strips whitespace from the location field and adds 1 point if the resulting string is empty.
- Classification uses chained threshold comparisons: score ≤ 2 yields 'Genuine', score ≤ 5 yields 'Suspicious', and score ≥ 6 yields 'Fake'.

5.1.3 Job Posting Implementation

- The /post_job route validates the session to confirm employer authentication before processing any POST data.
- Required fields are checked with Python's all() built-in, and email format is validated against a regex pattern.
- The salary field is safely converted to float with a try/except block, defaulting to None if conversion fails.
- The job record is inserted to the database with the employer's session user_id, the form data, and the computed score and classification.

5.1.4 Admin Dashboard Implementation

- The /admin/dashboard route queries all jobs with a JOIN on the employers table to retrieve employer details alongside each listing.
- An optional ?filter= query parameter limits results to a specific classification, implemented using conditional SQL query construction.

- A secondary query groups jobs by classification and counts them, producing a statistics dictionary passed to the template.
- The `/admin/delete_job/<int:job_id>` route executes a DELETE SQL statement for the specified job ID after verifying admin session status.

5.2 MODULES

- Module (Login, Registration, Logout)
- Employer Job Posting Module
- Fraud Detection and Scoring Module
- Job Listing Module (Employer View)
- Admin Dashboard Module

5.3 MODULE DESCRIPTION

AUTHENTICATION MODULE

Handles login and registration for both employer and admin account types. Validates credentials against hashed database records and establishes server-side sessions upon successful authentication.

EMPLOYER JOB POSTING MODULE

Provides the job submission form with validation, invokes the scoring algorithm, and stores the complete job record including the detection score and classification to the database.

FRAUD DETECTION MODULE

The `calculate_fake_score()` function implements four weighted rules and returns both the numeric score and the classification label for the submitted job data.

ADMIN DASHBOARD MODULE:

Queries all job postings across all employers, supporting optional filtering by classification. Displays a statistics summary and provides row-level delete actions.

5.4 POTENTIAL ENHANCEMENTS

5.4.1 Machine Learning Integration

- The rule-based scoring function can be augmented with a Naive Bayes or SVM classifier trained on the accumulated database of scored job postings.
- The existing database schema, which stores the detection score and classification for every job, naturally produces a labeled training dataset as the platform grows.

5.4.2 Real-Time Dashboard

- A live dashboard using WebSockets (Flask-SocketIO) could push real-time alerts to admin users when new Suspicious or Fake listings are submitted.

5.4.3 Email Notification System

- Automated email notifications to administrators when Fake listings are posted could reduce the time between submission and moderation action.

5.4.4 Advanced NLP Analysis

- Replacing the keyword-matching Rule 3 with a TF-IDF vectorizer and cosine similarity measure against a corpus of known fraudulent descriptions would improve detection of paraphrased scam language.

5.4.5 Employer Reputation Scoring

- Building a longitudinal reputation score for each employer based on their historical classification results would enable pre-emptive flagging of repeat offenders.

5.4.6 API Endpoints for Third-Party Integration

- Exposing a RESTful API would allow other job platforms to submit listings for classification without requiring direct database access or a user interface.

5.4.7 Cloud Deployment

- Deploying the application on AWS Elastic Beanstalk or Google Cloud Run with a managed MySQL instance would provide production-grade scalability and reliability.

5.4.8 Mobile Application

- A companion mobile application could allow administrators to review and moderate flagged listings directly from a smartphone.

CHAPTER 6

RESULTS & DISCUSSION

6.1 RESULTS

The Fake Job Detection System was successfully implemented and tested. The key results across all functional modules are presented below.

FRAUD DETECTION SCORING RESULTS

Job Scenario	Email (+2)	Salary (+2)	Keywords (+3)	Location (+1)	Result
Legitimate Corporate Job	0	0	0	0	Score: 0 – Genuine
Gmail Email + No Location	2	0	0	1	Score: 3 – Suspicious
Fee Required + High Salary	0	2	3	0	Score: 5 – Suspicious
Scam Post (All Triggers)	2	2	3	1	Score: 8 – Fake
Urgent Hiring + Gmail	2	0	3	0	Score: 5 – Suspicious
Max Fraud Indicators	2	2	3	1	Score: 8 – Fake

Table. 6.1 Fraud Detection Scoring Results

6.2 AUTHENTICATION MODULE RESULTS

- Employer registration successfully validates all fields, hashes passwords, and redirects to the login page upon success.

- Duplicate username/email detection works correctly, returning an error flash message without creating a duplicate record.
- Admin login correctly routes to the admin dashboard while employer login routes to the employer job list.
- Invalid credentials are rejected with an appropriate error message and no session is created.
- Logout successfully clears the session and prevents further access to protected routes.

6.3 ADMIN DASHBOARD RESULTS

- All posted jobs are displayed with employer name, classification badge, detection score, and submission timestamp.
- Filtering by Genuine / Suspicious / Fake correctly limits the displayed records to matching classifications only.
- The statistics panel correctly reflects the count of jobs in each classification category.
- The delete function successfully removes the selected job record from the database with a single admin action.

6.4 OUTPUT SCREENS

6.4.1 LOGIN SCREEN

The modern, dark-themed login interface for a **Fake Job Detection** system. The page features a centered login card with options to select account type (Employer or Admin), along with fields for username/email and password. A prominent login button and registration link are provided, all styled with a clean, minimal UI and soft gradient effects for a professional look.

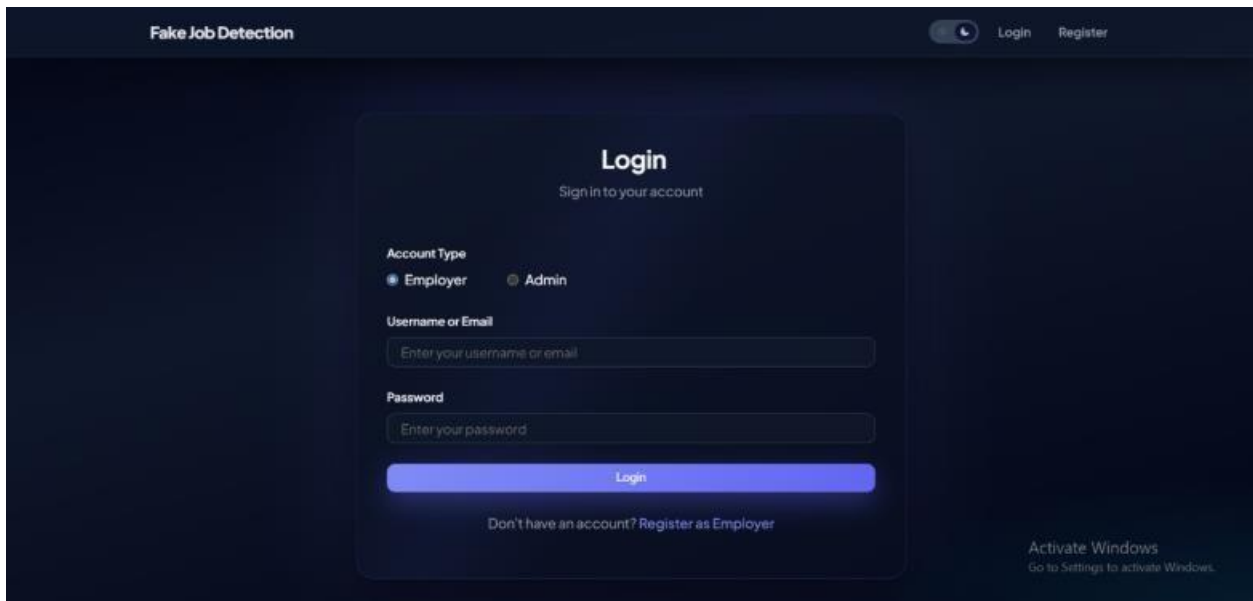


Fig. 6.1 Login Screen

6.4.2 EMPLOYEE REGISTER INTERFACE

The screen shows a **Register as Employer** page for the Fake Job Detection system. It features a clean, dark-themed form where users can create an employer account by entering details like username, email, company name, and password with confirmation. The interface is centered, minimal, and user-friendly, designed with soft gradients and modern styling for a professional appearance.

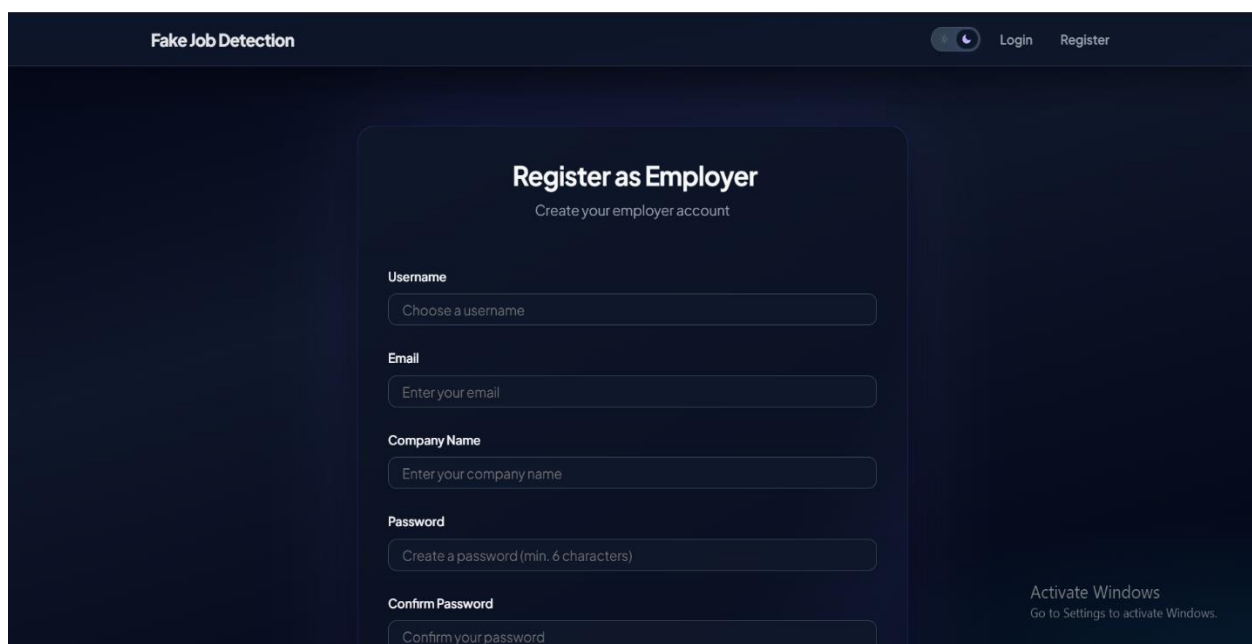


Fig. 6.2 Employee Register Screen

6.4.3 EMPLOYEE SCREEN

This screen represents the **Employer Dashboard** of the Fake Job Detection system. It provides an overview of posted jobs, detection scores, and hiring insights. Users can quickly post new jobs, view analytics, and monitor job authenticity through color-coded risk indicators, all within a modern, dark-themed interface designed for easy management and decision-making.

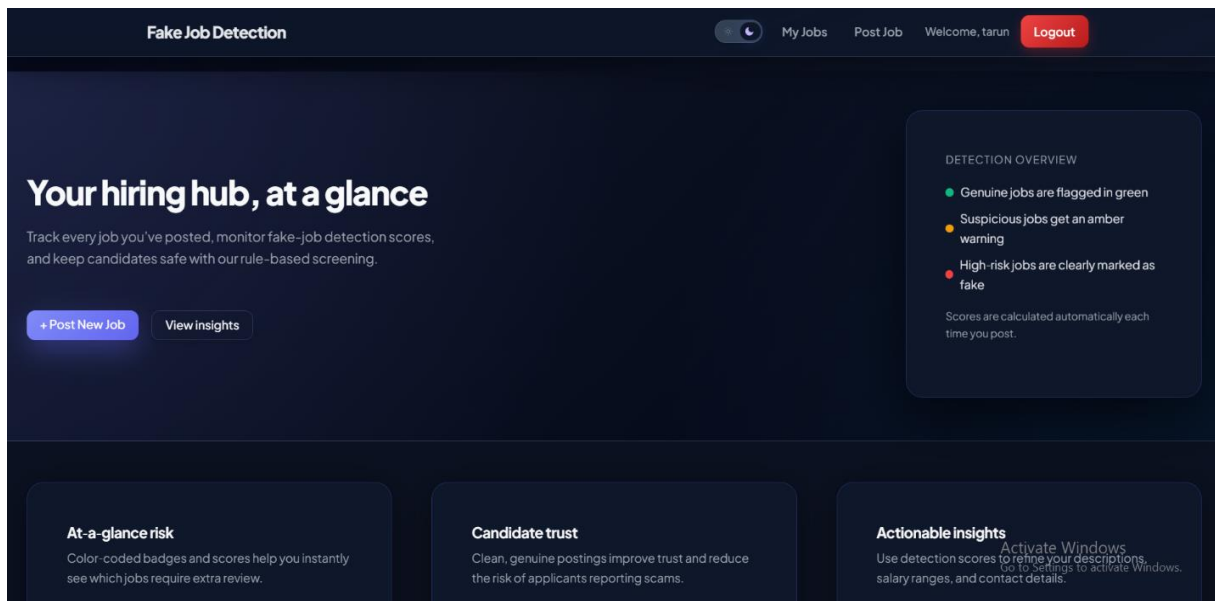


Fig. 6.3 Employee Screen

6.4.4 JOB POSTING SCREEN

This screen shows the **Post a New Job** page in the Fake Job Detection system. It provides a structured form where employers can enter details such as company name, email, job title, salary, location, and job description. The system automatically analyzes the submitted job for potential fraud indicators, offering a clean, dark-themed interface for easy and secure job posting.

Fake Job Detection My Jobs Post Job Welcome, tarun Logout

Post a New Job

Fill in the details below to post a new job listing. Our system will automatically analyze it for potential fraud indicators.

Company Name *
Enter company name

Email *
company@example.com

Job Title *
e.g., Software Engineer

Salary (\$)
e.g., 50000

Location
e.g., New York, NY

Job Description *
Describe the job responsibilities, requirements, and benefits...

Activate Windows
Go to Settings to activate Windows.

Fig. 6.4 Job Posting Screen

6.4.5 ADMIN DASHBOARD

This screen displays the **Job Listings Management** page in the Fake Job Detection system. It allows users to view and filter job postings based on classification (All, Genuine, Suspicious, Fake). Each job entry includes details like title, company, salary, location, detection score, and status, with color-coded labels for quick identification. Users can also manage listings with actions such as deleting jobs, all presented in a structured, dark-themed table interface.

Fake Job Detection Dashboard Welcome, ebshibha Logout

Filter by Classification

All Jobs Genuine Suspicious Fake

ID	Job Title	Company	Email	Salary	Location	Score	Classification	Employer	Posted Date	Actions
4	sql developer	tfcs	tfcs@gmail.com	\$10,000.00	Chennai	2	GENUINE	tarun	2026-04-11 22:49	Delete
3	hr position	ags solution	ags@gmail.com	\$150,000.00	banglore	7	FAKE	tarun	2026-02-28 13:28	Delete
1	figma designer	TFC Solutions	ebshibha@gmail.com	\$100,000.00	Chennai	5	SUSPICIOUS	tarun	2024-02-28 13:22	Delete

Activate Windows
Go to Settings to activate Windows.

Fig. 6.5 Admin Dashboard

6.4.6 RESULT OUTPUT

This screen shows the **My Job Postings** page in the Fake Job Detection system. It presents all posted jobs in a card-based layout, displaying details like company, salary, location, description, and contact information. Each job is clearly labeled as Genuine, Suspicious, or Fake with color-coded indicators and detection scores, allowing employers to quickly assess and manage their listings.

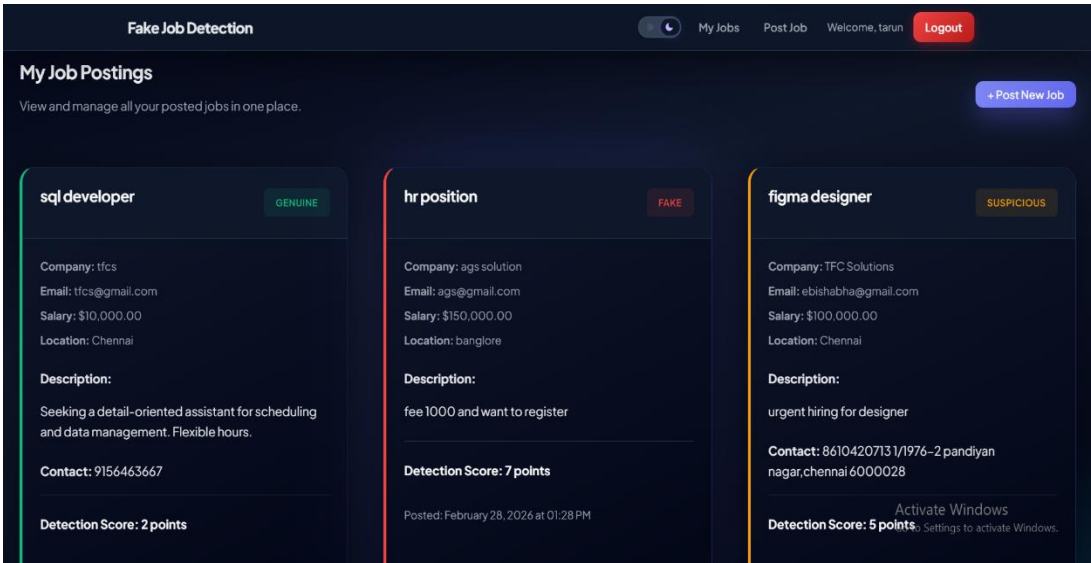


Fig. 6.6 Result Output

CHAPTER–7

CONCLUSION & FUTUREWORK

7.1 CONCLUSION

The growing reliance on online job platforms has brought forth new challenges in ensuring the authenticity of job postings and protecting job seekers from fraudulent listings. This project, the Fake Job Detection System, aimed to address this problem by developing a full-stack web application that automatically classifies job postings as Genuine, Suspicious, or Fake at the point of submission using a rule-based scoring algorithm.

The results demonstrated that the four-rule scoring system — evaluating email domain, salary level, suspicious keywords, and location completeness — effectively identifies fraudulent patterns across a range of test scenarios. The system successfully triages listings that a human administrator would need to review, enabling faster and more consistent moderation outcomes.

The dual-portal architecture, with separate interfaces for employers and administrators, provides a practical and usable platform for real-world deployment. Security best practices — including password hashing, parameterized SQL queries, and session-based authentication — are implemented throughout, ensuring the application meets baseline security standards for a production web service.

In conclusion, the Fake Job Detection System demonstrates the viability of rule-based heuristics as a real-time, interpretable, and low-latency first line of defense against online employment fraud. The system's modular architecture provides a clear pathway for future enhancement through machine learning integration, advanced NLP analysis, and cloud-scale deployment.

7.2 FUTURE SCOPE

The Fake Job Detection System lays a strong foundation for automated employment fraud prevention. However, as the landscape of fraudulent activity evolves and the volume of job submissions grows, so too must the tools used to detect

them. Future developments in this system aim to enhance accuracy, scalability, and operational responsiveness through the following directions.

The most significant planned enhancement is the integration of a machine learning classifier trained on the database of scored and classified job postings accumulated by the live system. As the dataset grows, a Naive Bayes, SVM, or finetuned BERT model could replace or augment the rule-based scoring function, handling edge cases that fall between classification thresholds more accurately. The existing `detection_score` and `classification` columns in the `jobs` table provide a readymade labeled dataset for this purpose.

Advanced Natural Language Processing could improve Rule 3 by replacing simple keyword matching with TF-IDF vectorization and semantic similarity measures, detecting paraphrased scam language that avoids the exact trigger keywords. Incorporating Named Entity Recognition to identify and validate company names and locations would further strengthen detection accuracy.

Operationally, a real-time notification dashboard using WebSockets and a mobile administration application would enable faster response to newly flagged listings. Integration with third-party employer verification APIs would add a supplementary data source to the scoring process. Cloud deployment on AWS or Google Cloud Platform would provide the scalability needed to serve a large, multitenant job platform with consistent performance under high submission volume.

APPENDIX

App.py

"""

Fake Job Detection System - Flask Application

=====

Main Flask application file containing all routes and business logic.

Handles authentication, job posting, fake detection, and admin dashboard. """

from flask import Flask, render_template, request, redirect, url_for, session, flash, jsonify

from werkzeug.security import generate_password_hash, check_password_hash

from datetime import datetime import re

from config.database import get_db, close_db, init_db

Initialize Flask application app = Flask(__name__) app.secret_key = 'your-secret-key-change-this-in-production' # Change this in production!

Register database teardown handler

app.teardown_appcontext(close_db)

=====

Rule-Based Fake Job Detection Algorithm

=====

def calculate_fake_score(job_data):

"""

Calculate fake job detection score based on predefined rules.

Scoring Rules:

- Gmail/Yahoo email addresses: +2 points
- Salary exceeding \$100,000: +2 points
- Job description contains "fee", "registration", or "urgent hiring": +3 points
- Empty location field: +1 point

Classification Thresholds:

- 0-2 points: Genuine
- 3-5 points: Suspicious
- 6+ points: Fake

Args:

job_data (dict): Dictionary containing job posting data with keys:

- email: Employer email address
- salary: Job salary (float or string)
- job_description: Job description text
- location: Job location (string)

Returns: tuple: (score (int),
classification (str))

"""

score = 0

Rule 1: Check for Gmail/Yahoo email addresses (+2 points)

email = job_data.get('email', '').lower() if

'@gmail.com' in email or '@yahoo.com' in email:

score += 2

Rule 2: Check if salary exceeds \$100,000 (+2 points)

try:

salary = float(job_data.get('salary', 0))

if salary > 100000:

score += 2 except

(ValueError, TypeError):

pass # If salary is invalid, skip this rule

Rule 3: Check job description for suspicious keywords (+3 points)

job_description = job_data.get('job_description', '').lower()

suspicious_keywords = ['fee', 'registration', 'urgent hiring'] if

any(keyword in job_description for keyword in suspicious_keywords):

```

        score += 3

    # Rule 4: Check if location field is empty (+1 point)
    location = job_data.get('location', "").strip()    if not
location:        score += 1

    # Determine classification based on score
    if score <= 2:
        classification = 'Genuine'
    elif score <= 5:
        classification = 'Suspicious'
    else:        #    score    >=    6
        classification = 'Fake'

    return score, classification

@app.route('/control-center')
def control_center():    if
'user_id' not in session:
    return redirect(url_for('login'))

    try:
        db = get_db()
        cur = db.cursor(dictionary=True)
        cur.execute("SELECT    COUNT(*)    AS    c    FROM    jobs")
jobs_count = cur.fetchone()['c']
        cur.execute("SELECT COUNT(*) AS c FROM jobs WHERE detection_score
>= 6")
        high_risk_count = cur.fetchone()['c']
        cur.execute("SELECT COUNT(*) AS c FROM jobs WHERE classification =
'Suspicious'")
        suspicious_count = cur.fetchone()['c']

```

```

        suspicious_rate = f'{round((suspicious_count / jobs_count) * 100)}%' if
jobs_count else "0%"
        cur.execute(
            "SELECT * FROM jobs ORDER BY created_at DESC LIMIT 30"
        )
        jobs = cur.fetchall()
    cur.close() except
Exception:      jobs_count
= 0      high_risk_count =
0      suspicious_rate =
"0%"      jobs = []

    return render_template(
'saas_shell.html',      jobs=jobs,
        jobs_count=jobs_count,
high_risk_count=high_risk_count,
suspicious_rate=suspicious_rate,
    )
@app.route('/') def
index():
return redirect(url_for('login'))
@app.route('/login', methods=['GET', 'POST']) def
login():
    """
    Handle user login (both employers and admins).
    Supports both regular form submission and AJAX requests.
    """
    if request.method == 'POST':

```

```

        username    =    request.form.get('username',    ").strip()
password = request.form.get('password', ").strip()
        user_type = request.form.get('user_type', 'employer') # 'employer' or 'admin'
        if not username or not password:
            flash('Please fill in all fields', 'error')
if request.is_json:
        return jsonify({'success': False, 'message': 'Please fill in all fields'}), 400
return render_template('login.html')
try:
        db = get_db()
        cursor = db.cursor(dictionary=True)
        # Query appropriate table based on user type
if user_type == 'admin':
        cursor.execute(
            "SELECT * FROM admins WHERE username = %s OR email = %s",
            (username, username)
        )
else:
        cursor.execute(
            "SELECT * FROM employers WHERE username = %s OR email =
%s",
            (username, username)
        )
        user    =    cursor.fetchone()
cursor.close()
        if user and check_password_hash(user['password_hash'], password):
session['user_id'] = user['id']
session['username'] = user['username']
session['user_type'] = user_type

```

```

        if request.is_json:
return jsonify({
                'success': True,
                'message': 'Login successful',
                'redirect': url_for('admin_dashboard' if user_type == 'admin' else
'job_list')
            })
if user_type == 'admin':
    return redirect(url_for('admin_dashboard'))
else:
    return redirect(url_for('job_list'))
else:
    flash('Invalid username or password', 'error')
if request.is_json:
    return jsonify({'success': False, 'message': 'Invalid username or
password'}), 401
    except Exception as e:
        flash(f'Database error: {str(e)}', 'error')
        if request.is_json:
            return
jsonify({'success': False, 'message': 'Database error occurred'}), 500
    return render_template('login.html')
@app.route('/register', methods=['GET', 'POST']) def
register():
    """
    Handle employer registration.
    Creates new employer account with password hashing.
    """
    if request.method == 'POST':
        username = request.form.get('username', "").strip()
email = request.form.get('email', "").strip()
password = request.form.get('password', "").strip()

```

```

confirm_password = request.form.get('confirm_password', "").strip()
company_name = request.form.get('company_name', "").strip()
if not all([username, email, password, confirm_password, company_name]):
    flash('Please fill in all fields', 'error')
if request.is_json:
    return jsonify({'success': False, 'message': 'Please fill in all fields'}), 400
    return render_template('register.html')
if password != confirm_password:
    flash('Passwords do not match', 'error')
if request.is_json:
    return jsonify({'success': False, 'message': 'Passwords do not match'}), 400
return render_template('register.html')
    if len(password) < 6:
        flash('Password must be at least 6 characters long', 'error')
if request.is_json:
    return jsonify({'success': False, 'message': 'Password must be at least 6
characters'}), 400
    return render_template('register.html')
    email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
if not re.match(email_pattern, email):
    flash('Invalid email format',
'error')
    if request.is_json:
        return jsonify({'success': False, 'message': 'Invalid email format'}), 400
return render_template('register.html')
try:
    db = get_db()
cursor = db.cursor()
cursor.execute(
    "SELECT id FROM employers WHERE username = %s OR email = %s",
    (username, email)

```

```

    )
    if cursor.fetchone():
        flash('Username or email already exists', 'error')
    cursor.close()
    if request.is_json:
        return jsonify({'success': False, 'message': 'Username or email already
exists'}), 400
    return render_template('register.html')

password_hash = generate_password_hash(password)
cursor.execute(
    """INSERT INTO employers (username, email, password_hash,
company_name)
    VALUES (%s, %s, %s, %s)""",
    (username, email, password_hash, company_name)
)
db.commit()
cursor.close()
flash('Registration successful! Please login.', 'success')
if request.is_json:
    return jsonify({
        'success': True,
        'message': 'Registration successful',
        'redirect': url_for('login')
    })
    return redirect(url_for('login'))
except Exception as e:
    flash(f'Database error: {str(e)}', 'error')
    if request.is_json:
        return
    jsonify({'success': False, 'message': 'Database error occurred'}), 500
    return render_template('register.html')

```



```

@app.route('/logout') def
logout():
    """Handle user logout"""
    session.clear()
    flash('You have been logged out', 'info')
    return redirect(url_for('login'))
@app.route('/post_job', methods=['GET', 'POST']) def
post_job():
    if 'user_id' not in session or session.get('user_type') != 'employer':
        flash('Please login as an employer to post jobs', 'error')
    return redirect(url_for('login'))
    if request.method == 'POST':
        company_name = request.form.get('company_name', "").strip()
        email = request.form.get('email', "").strip()
        job_title = request.form.get('job_title', "").strip()
        salary = request.form.get('salary', "").strip()
        location = request.form.get('location', "").strip()
        job_description = request.form.get('job_description', "").strip()
        contact_details = request.form.get('contact_details', "").strip()
        if not all([company_name, email, job_title, job_description]):
            flash('Please fill in all required fields', 'error')
        if request.is_json:
            return jsonify({'success': False, 'message': 'Please fill in all required
fields'}), 400
        return render_template('post_job.html')
        email_pattern = r'^[a-zA-Z0-9._%+~]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
        if not re.match(email_pattern, email):
            flash('Invalid email format',
'error')
            if request.is_json:
                return jsonify({'success': False, 'message': 'Invalid email format'}), 400

```

```

        return render_template('post_job.html')
try:
    salary_float = float(salary) if salary else None
except ValueError:
    salary_float = None
job_data = {
    'email': email,
    'salary': salary_float,
    'job_description': job_description,
    'location': location
}
score, classification = calculate_fake_score(job_data)
try:
    db = get_db()
    cursor = db.cursor()
    cursor.execute(
        """INSERT INTO jobs (employer_id, company_name, email, job_title,
salary,
        location,    job_description,    contact_details,    detection_score,
classification)
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)""",
        (session['user_id'], company_name, email, job_title, salary_float,
        location, job_description, contact_details, score, classification)
    )
    db.commit()
    cursor.close()
    flash(f'Job posted successfully! Classification: {classification} (Score:

```

```

{score})),      'success')
if      request.is_json:
return jsonify({
            'success': True,
            'message': f'Job posted successfully! Classification: {classification}',
'score': score,
            'classification': classification
        })
        return redirect(url_for('job_list'))
except Exception as e:
        flash(f'Database error: {str(e)}', 'error')      if request.is_json:      return
        jsonify({'success': False, 'message': 'Database error occurred'}), 500
        return render_template('post_job.html')
@app.route('/jobs') def
job_list():
if 'user_id' not in session:
        flash('Please login to view jobs', 'error')
return redirect(url_for('login'))
try:
        db = get_db()
        cursor = db.cursor(dictionary=True)
cursor.execute(
        """SELECT j.*, e.company_name as employer_company
        FROM jobs j
        JOIN employers e ON j.employer_id = e.id
        WHERE j.employer_id = %s
        ORDER BY j.created_at DESC""",
        (session['user_id'],)
    )

```

```

        jobs            =            cursor.fetchall()
cursor.close()

        return render_template('job_list.html', jobs=jobs)
except Exception as e:
    flash(f'Database error: {str(e)}', 'error')
return render_template('job_list.html', jobs=[]) #
@app.route('/admin/dashboard') def
admin_dashboard():
    if 'user_id' not in session or session.get('user_type') != 'admin':
        flash('Access denied. Admin login required.', 'error')
return redirect(url_for('login'))
    filter_classification = request.args.get('filter', 'all')
    try:
        db = get_db()
        cursor = db.cursor(dictionary=True)
if filter_classification == 'all':
        cursor.execute(
            """SELECT j.id, j.employer_id, j.company_name, j.email, j.job_title,
j.salary,
                j.location, j.job_description, j.contact_details, j.detection_score,
                j.classification, j.created_at, j.updated_at,
                e.username as employer_username, e.company_name as
employer_company
                FROM jobs j
                JOIN employers e ON j.employer_id = e.id
                ORDER BY j.created_at DESC"""
        )
    else:
        cursor.execute(

```

```

        """SELECT j.id, j.employer_id, j.company_name, j.email, j.job_title, j.salary,
               j.location, j.job_description, j.contact_details, j.detection_score,
               j.classification, j.created_at, j.updated_at,
               e.username as employer_username, e.company_name as
employer_company
        FROM jobs j
        JOIN employers e ON j.employer_id = e.id
        WHERE j.classification = %s
        ORDER BY j.created_at DESC""",
        (filter_classification,)
    )
    jobs = cursor.fetchall()

cursor.execute(
    "SELECT classification, COUNT(*) as count FROM jobs GROUP BY
classification"
)
stats = cursor.fetchall()
stats_dict = {stat['classification']: stat['count'] for stat in stats}
cursor.close()

return render_template('admin_dashboard.html',
                       jobs=jobs,
                       filter_classification=filter_classification,
stats=stats_dict)

except Exception as e:
    flash(f'Database error: {str(e)}', 'error')

    return render_template('admin_dashboard.html', jobs=[], stats={})

@app.route('/admin/delete_job/<int:job_id>', methods=['POST']) def
delete_job(job_id):

```

```

    if 'user_id' not in session or session.get('user_type') != 'admin':
if request.is_json:
    return jsonify({'success': False, 'message': 'Access denied'}), 403
flash('Access denied', 'error')    return redirect(url_for('login'))

    try:
        db    =    get_db()
cursor = db.cursor()

        cursor.execute("DELETE FROM jobs WHERE id = %s", (job_id,))
db.commit()    cursor.close()

        flash('Job deleted successfully', 'success')
if request.is_json:
    return jsonify({'success': True, 'message': 'Job deleted successfully'})
    return redirect(url_for('admin_dashboard'))

except Exception as e:
    flash(f'Database error: {str(e)}', 'error')
if request.is_json:
    return jsonify({'success': False, 'message': 'Database error occurred'}), 500
return redirect(url_for('admin_dashboard'))

if __name__ == '__main__':
try:
    init_db()    except
Exception as e:
    print(f'Warning: Could not initialize database: {e}')
    print("Make sure MySQL is running and database credentials are correct.")
app.run(debug=True, host='0.0.0.0', port=5000)

```

REFERENCES

1. Ahmed .T, S. Islam, and M. Hossain, "Automated Screening of Fake Job Postings Using Lexical Rule Sets and Boolean Rule Chains," in Proc. Int. Conf. on Computing, Communication, and Intelligent Systems (ICCCIS), Greater Noida, India, 2023, pp. 412–419.
2. Alghamdi .B and N. Alrajeh, "An Explainable Rule Engine for Detecting Fake Employment Advertisements Using Hierarchical Rule Trees," IEEE Access, vol. 11, pp. 62741–62755, Jun. 2023.
3. EMSCAD – Employment Scam Aegean Dataset, University of the Aegean, Mytilene, Greece. [Online]. Available: <http://emscad.samos.aegean.gr>. [Accessed: Mar. 2026].
4. Frank .H and T. Brill, "Lightweight Text Screening for Employment Fraud Detection in Low-Resource Environments," ACM SIGKDD Explorations Newsletter, vol. 24, no. 1, pp. 45–58, Jun. 2022.
5. Qureshi .M .A, A. Sabir, and F. Bashir, "Job Scam Detection via Structural Metadata Rule Checking: A Field Completeness Approach," in Proc. IEEE Int. Conf. on Emerging Trends in Information Technology and Engineering (ICETITE), Vellore, India, 2021, pp. 1–6.
6. Qureshi .M .A, A. Sabir, and F. Bashir, "Job Scam Detection via Structural Metadata Rule Checking: A Field Completeness Approach," in Proc. IEEE Int. Conf. on Emerging Trends in Information Technology and Engineering (ICETITE), Vellore, India, 2021, pp. 1–6.
7. Raj .D and N. Anand, "A Comparative Study of Keyword-Based and Structural Rule Systems for Fake Job Posting Detection," International Journal of Computer Applications, vol. 184, no. 28, pp. 1–8, Dec. 2022.
8. Rao .S and B. Prasad, "Real-Time Fake Job Filtering Using Pattern-Matched Rule Sets on Job Portal APIs," in Proc. IEEE Int. Conf. on Intelligent Data

- Science Technologies and Applications (IDSTA), Valencia, Spain, 2024, pp. 88–95.
9. Sarker .I .H, "Data Science and Analytics: An Overview from Data-Driven Smart Computing, Decision-Making and Applications Perspective," SN Computer Science, vol. 2, no. 5, p. 377, Sep. 2021.
 10. Sawhney .R, S. Agarwal, A. Wadhwa, and R. R. Shah, "Exploring Textual Indicators of Recruitment Fraud: A Corpus-Based Study on the EMSCAD Dataset," in Proc. Annual Conf. of the North American Chapter of the Association for Computational Linguistics (NAACL), Online, 2021, pp. 2834–2845.
 11. Scarfone .K and P. Mell, "Guide to Intrusion Detection and Prevention Systems (IDPS)," NIST Special Publication 800-94, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2007.
 12. Vidros .S, C. Koliass, G. Kambourakis, and L. Akoglu, "Automatic Detection of Online Recruitment Frauds: Characteristics, Methods, and a Public Dataset," Future Internet, vol. 9, no. 1, p. 6, Mar. 2017.
 13. Von Solms .R and J. van Niekerk, "From Information Security to Cyber Security," Computers & Security, vol. 38, pp. 97–102, Oct. 2013.
 14. Whitecomb .L, A. Patel, and R. Nguyen, "A Hybrid Rule Framework for Employment Scam Identification Using Keyword Blacklists and Metadata Completeness Scoring," Journal of Information Security and Applications, vol. 65, pp. 103–118, Mar. 2022.
 15. Whitecomb .L and G. Stern, "Keyword Coverage and Recall in Rule-Based Employment Fraud Filters: A Lexicon Construction Methodology," Computers & Security, vol. 112, p. 102504, Jan. 2022.

